

WHIR on Bitcoin Script

A formal account of the STIR and WHIR proximity tests,
and a review of bitcoin-stark-verifier against them

Subject: AppliedPQC/bitcoin-stark-verifier · Sources: ePrint 2024/390, 2024/1586 · 98
tests passing · D1–D3 fixed; queries composed, threaded and closed

SUMMARY OF FINDINGS

- **The primitives are sound and genuinely verified.** Poseidon2, the duplex sponge, the challenger, the sumcheck round and multilinear evaluation are each checked against a Rust transcription of Plonky3, with negative cases. A real Plonky3 WHIR proof’s Merkle opening authenticates in script, and a tampered leaf is rejected.
- **Three defects sat above them**, all in the composition rather than the arithmetic: Fiat–Shamir challenges were never derived (**D1**), the closing identity was a tautology (**D2**), and the composed `verify()` was a cost model that was never executed (**D3**). All three are fixed, with adversarial tests.
- **The cost is hashing, and almost nothing else.** The example configuration is 2162 permutations, of which **97.1%** is query work — Merkle paths and the leaf hash that binds a row to them — against 2.9% for the whole transcript.
- **The verifier ends on its identity, and derives everything in it.** The queries are composed, their roots bound to the absorbed commitment, the folding randomness accumulated, and the closing check `claimed = w(R) · fM(rfin)` emitted with its constraints *derived from the transcript* rather than supplied — for 7.5% of the verifier and no permutation at all. Only the statement is supplied, because a statement is what a claim *is*.
- **Two configuration levers remain unused**, worth roughly 20% of script weight between them: `pow_bits = 0` where both reference implementations use ≈ 22 , and the choice of soundness regime, currently the conjectural one.

Part I The algorithm, formally

1 Setting and notation

Fix a finite field $\mathbb{F}$ with $\text{char}(\mathbb{F}) \neq 2$ and a *smooth* evaluation domain $\mathcal{L} \subseteq \mathbb{F}^*$ — a multiplicative coset whose order is a power of two. For a degree bound d ,

$$\text{RS}[\mathbb{F}, \mathcal{L}, d] = \{ f : \mathcal{L} \rightarrow \mathbb{F} \mid \exists \hat{g} \in \mathbb{F}^{<d}[X], f(x) = \hat{g}(x) \ \forall x \in \mathcal{L} \}, \quad \rho := d/|\mathcal{L}|.$$

When $d = 2^m$ the code has an equivalent multilinear reading, and this is the view both protocols use. Writing $\text{pow}(x, m) = (x^{2^0}, \dots, x^{2^{m-1}})$,

$$\text{RS}[\mathbb{F}, \mathcal{L}, m] = \{ f \mid \exists \hat{f} \in \mathbb{F}^{<2}[X_1, \dots, X_m], f(x) = \hat{f}(\text{pow}(x, m)) \}.$$

$\Delta(f, g)$ is fractional Hamming distance and $\Lambda(\mathcal{C}, f, \delta)$ the list. The equality polynomial is

$$\text{eq}(\mathbf{X}, \mathbf{Y}) = \prod_{i=1}^m (X_i Y_i + (1 - X_i)(1 - Y_i)), \quad \hat{f}(\mathbf{z}) = \sum_{\mathbf{b} \in \{0,1\}^m} \hat{f}(\mathbf{b}) \cdot \text{eq}(\mathbf{b}, \mathbf{z}).$$

2 Constrained Reed–Solomon codes

WHIR’s object of study is not RS but a subcode carrying one sumcheck-shaped constraint:

$$\text{CRS}[\mathbb{F}, \mathcal{L}, m, \hat{w}, \sigma] = \left\{ f \in \text{RS}[\mathbb{F}, \mathcal{L}, m] : \sum_{\mathbf{b} \in \{0,1\}^m} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \sigma \right\}. \quad (\text{Def. 1})$$

The weight polynomial

Definition 1 leans on $\hat{w}$ without saying what it is, and it is worth being precise, because every later step is a statement about $\hat{w}$ and almost all of the implementation work is bookkeeping for it.

$\hat{w}$ is a polynomial in $1 + m$ variables,

$$\hat{w}(Z, \mathbf{X}) = \hat{w}(Z, X_1, \dots, X_m),$$

whose first argument Z is a *placeholder for a value of the polynomial being tested* and whose remaining arguments $\mathbf{X}$ range over positions on the hypercube. Substituting $Z \mapsto \hat{f}(\mathbf{b})$ and $\mathbf{X} \mapsto \mathbf{b}$ and summing over $\mathbf{b} \in \{0,1\}^m$ turns it into a single field element, and Definition 1 asks that element to be σ . So $\hat{w}$ is not a constraint on f pointwise — it is the *shape* of one aggregate linear statement about f , and σ is its claimed value.

Two restrictions make it usable rather than merely general.

(i) **Linear in Z .** Every $\hat{w}$ WHIR forms has the shape

$$\hat{w}(Z, \mathbf{X}) = Z \cdot w(\mathbf{X}), \quad w \text{ multilinear in } \mathbf{X},$$

so the sum $\sum_{\mathbf{b}} \hat{f}(\mathbf{b}) w(\mathbf{b})$ is a sumcheck over a *product of two multilinear*s. That is what fixes the degree of every round polynomial at two, which in turn is why a round sends three values — and why, with $\hat{h}(0) + \hat{h}(1) = \sigma$ pinning one of them, only $\hat{h}(0)$ and $\hat{h}(\infty)$ travel. The entire cost model of §9 follows from this one line.

(ii) **Closed under the recursion.** §6’s update

$$\hat{w}'(Z, \mathbf{X}) = \hat{w}(Z, \boldsymbol{\alpha}, \mathbf{X}) + Z \cdot \sum_{i=0}^t \gamma^{i+1} \text{eq}(\mathbf{z}_i, \mathbf{X})$$

is linear in Z whenever $\hat{w}$ is, since binding $\boldsymbol{\alpha}$ does not touch Z and the added term is visibly Z times something. So $\hat{w}'$ is again of the form $Z \cdot w'(\mathbf{X})$, and the protocol never leaves the class it started in.

Unfolding (ii) across all M rounds gives the concrete object the verifier ends up holding:

$$w(\mathbf{X}) = \sum_j c_j \cdot \text{eq}(\mathbf{z}_j, \mathbf{X}),$$

a **weighted sum of equality polynomials** — that is, nothing more than a list of pairs $(c_j, \mathbf{z}_j)$. Every constraint the protocol ever accumulates, from the initial evaluation query through each round’s out-of-domain answer and t shift answers, is one entry in that list, and the coefficients c_j are powers of the batching challenges. §14 is about carrying that list, and §15 is about what it costs to evaluate it.

Evaluation as a special case

The point of the definition is that a polynomial *evaluation query* is a special case. Taking $\hat{w}(Z, \mathbf{X}) = Z \cdot \text{eq}(\mathbf{X}, \mathbf{z})$ gives $\sum_{\mathbf{b}} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \hat{f}(\mathbf{z})$, so membership with that weight *is* the claim $\hat{f}(\mathbf{z}) = \sigma$. A proximity test for CRS codes is therefore simultaneously a low-degree test and a multilinear polynomial commitment — which is why WHIR replaces FRI, STIR and BaseFold at once rather than sitting beside them.

3 Folding

For $\alpha \in \mathbb{F}$ and $y = x^2$,

$$\text{Fold}(f, \alpha)(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x},$$

extended to $\alpha = (\alpha_1, \dots, \alpha_k)$ by iteration. The structural fact is that folding is multilinear partial evaluation seen through the code: if $f \in \text{RS}[\mathbb{F}, \mathcal{L}, m]$ then

$$\text{Fold}(f, \alpha) \in \text{RS}[\mathbb{F}, \mathcal{L}^{(2^k)}, m - k], \quad \hat{g}(X_{k+1}, \dots, X_m) = \hat{f}(\alpha, X_{k+1}, \dots, X_m). \quad (\text{Claim 4.15})$$

Three quantities move together, and it is worth reading $m - k$ as the count it is: binding the first k variables to α leaves $X_{k+1}, \dots, X_m$ free, so the arity falls from m to $m - k$, the degree from 2^m to 2^{m-k} , and the domain from $\mathcal{L}$ to $\mathcal{L}^{(2^k)}$ of size $|\mathcal{L}|/2^k$.

That third movement is the one to watch, because of what it does to the rate:

$$\rho = \frac{2^m}{|\mathcal{L}|} \quad \longrightarrow \quad \rho' = \frac{2^{m-k}}{|\mathcal{L}|/2^k} = \frac{2^m}{|\mathcal{L}|} = \rho.$$

Folding alone does not change the rate. Claim 4.15 is the structural half of the story and says nothing about cost. The rate reduction of §4 comes from somewhere else entirely — from the prover re-committing g on a domain that is merely *halved* rather than divided by 2^k — which is why that section is about a choice and this one is about an identity.

4 Rate is the design variable

This is the whole idea behind both protocols, and it is a single line of arithmetic.

Protocol	Degree	Domain	Rate	$\log_2(1/\rho)$ per round
FRI	$d \rightarrow d/k$	$ \mathcal{L} \rightarrow \mathcal{L} /k$	$\rho' = \rho$	unchanged
STIR	$d \rightarrow d/k$	$ \mathcal{L} \rightarrow \mathcal{L} /2$	$\rho' = (2/k)\rho$	$+\log_2 k - 1$
WHIR	$m \rightarrow m - k$	$ \mathcal{L} \rightarrow \mathcal{L} /2$	$\rho' = 2^{1-k}\rho$	$+(k - 1)$

A notational trap. STIR's k is the degree *divisor* (a power of two, $k \geq 4$); WHIR's k is the *number of variables* folded. The two agree under $k_{\text{STIR}} = 2^{k_{\text{WHIR}}}$. The recurrence that appears in code is WHIR's.

The number of queries needed in round i to reach round-by-round error $2^{-\lambda}$ is $t_i = \lceil \lambda / -\log_2(\rho_i + \eta_i) \rceil$ with $\rho_i = 2^{-i(k-1)}\rho$, so $t_i \approx \lambda / (\log_2(1/\rho) + i(k-1))$. The sum over $M = m/k$ rounds is harmonic, not linear; using $\sum_{i=1}^M 1/(i+c) < \log(M/c+1) + 1$ (STIR Fact C.1),

$$q = O\left(\frac{\lambda}{\log(1/\rho)} + \frac{\lambda}{k} \log\left(\frac{m}{k \log(1/\rho)}\right) + \frac{m}{k}\right) \quad \text{versus FRI's} \quad O\left(\lambda + \frac{\lambda m}{k}\right).$$

In the typical regime $m \leq \lambda$, $\rho = O(1)$, setting $k \approx \log m$ yields $q = O(\lambda)$, which is optimal. **There is no further protocol-level lever on query count.**

5 A STIR iteration: quotient and degree correction

STIR sends a folded g on a larger domain, pins it with an out-of-domain sample, checks it against t shift queries, and forms the next function as a quotient:

$$\text{Quotient}(f, S, \text{Ans}, \text{Fill})(x) = \frac{f(x) - \widehat{\text{Ans}}(x)}{\widehat{V}_S(x)} \quad (x \notin S).$$

Two of those symbols carry the whole mechanism. $S = \{z_0, z_1, \dots, z_t\}$ is the out-of-domain sample together with the shift points; $\widehat{\text{Ans}}$ interpolates the values the verifier computed *for itself* at those points; and

$$\widehat{V}_S(X) = \prod_{z \in S} (X - z)$$

is the *vanishing polynomial* of S — monic, of degree $|S| = t + 1$, zero exactly on S .

The quotient is a polynomial only when the numerator vanishes wherever the denominator does, so

$$\begin{aligned} \frac{g - \widehat{\text{Ans}}}{\widehat{V}_S} \text{ is a polynomial} &\iff g - \widehat{\text{Ans}} \text{ vanishes on } S \\ &\iff g \text{ carries the verifier's answers at } S. \end{aligned}$$

That equivalence *is* the enforcement. Nobody checks the answers directly; the division either comes out clean or it does not, and when it does not the result is not slightly wrong but far from low degree (Lemma 4.4) — so the next round's proximity test, which was running anyway, rejects it. A lie is converted into a distance the existing machinery already measures.

Two consequences follow from $\widehat{V}_S$ alone. The side condition $x \notin S$ is not fussiness: the denominator vanishes there, which is why the new domain must be chosen disjoint from S and why Fill exists to supply values at the holes. And the degree drops by $|S|$, so the round closes with degree correction — a geometric sum evaluated in $O(\log e)$ operations by repeated squaring rather than term by term.

6 A WHIR iteration: constraints instead of quotients

1. **Sumcheck rounds.** k rounds on $\sum_{\mathbf{b}} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \sigma$, giving $\hat{h}_{1..k}$ and randomness α .
2. **Folded function.** The prover sends g on $\mathcal{L}^{(2)}$.
3. **Out-of-domain sample.** $z_0 \leftarrow \mathbb{F}$, reply y_0 .
4. **Shift queries.** $z_1, \dots, z_t \leftarrow \mathcal{L}^{(2^k)}$, answers $y_i = \text{Fold}(f, \alpha)(z_i)$; then a batching challenge γ .
5. **Recursive claim.**

$$\begin{aligned} \hat{w}'(Z, \mathbf{X}) &:= \hat{w}(Z, \alpha, \mathbf{X}) + Z \cdot \sum_{i=0}^t \gamma^{i+1} \cdot \text{eq}(z_i, \mathbf{X}), \\ \sigma' &:= \hat{h}_k(\alpha_k) + \sum_{i=0}^t \gamma^{i+1} \cdot y_i. \end{aligned}$$

The out-of-domain answer and the t shift answers become $t + 1$ additional *constraints*, folded into the weight polynomial by powers of γ . No quotient, hence no degree correction, hence — as the paper states explicitly — **the verifier performs no divisions at all**. On a machine without OP_DIV, where an inversion must be hinted and checked, that is the reason this project should target WHIR rather than STIR.

7 What a shift query actually is

The most frequently occurring and most easily skimmed quantity in the protocol is r_i^{shift} (WHIR writes z_i). It deserves its own section, because in this implementation it *is* “a query”, and the identification is not obvious.

Which domain it lives in

The first thing to get straight: a shift query is **not** drawn from the new domain $\mathcal{L}'$ the prover just committed to. It is drawn from

$$r_i^{\text{shift}} \leftarrow \mathcal{L}^{(2^k)} = \{x^{2^k} : x \in \mathcal{L}\}, \quad |\mathcal{L}^{(2^k)}| = |\mathcal{L}|/2^k,$$

the domain the *folded* function lives on. And the folded function is **virtual** — no one sends it. To read it at a point the verifier must read the original f across the whole fibre:

$$y_i = \text{Fold}(f, \alpha)(r_i^{\text{shift}}), \quad \text{requiring } f \text{ on } \{x : x^{2^k} = r_i^{\text{shift}}\}, \text{ which is } 2^k \text{ values.}$$

Those 2^k values are the paper’s “alphabet $\mathbb{F}^{2^k}$ ” — one query reads one symbol, and a symbol is 2^k field elements. In the implementation it is a Merkle **row**, and it is exactly the row that has to be bound to its leaf.

Why it has to exist

This is the pivot of the whole STIR/WHIR construction, and the rate reduction of §4 is what it pays for. In FRI the folded function lives *naturally* on $\mathcal{L}^{(k)}$ and the verifier can compute it unaided. To lower the rate, STIR and WHIR have the prover re-commit a g on a *larger* new domain.

But nothing forces g to be $\text{Fold}(f, \alpha)$. The prover could send an unrelated low-degree polynomial. **The shift queries are the pin that fastens the new oracle back to the old one:** they require $\hat{g}(r_i^{\text{shift}}) = \text{Fold}(f, \alpha)(r_i^{\text{shift}})$ at t random points. Hence the name: STIR = *Shift To Improve Rate*. Out-of-domain sampling does a different job — it pins g to a *unique* codeword in the list; the shift queries then check that this codeword is the fold. Neither suffices alone.

It is the soundness term

t is not arbitrary; it is precisely the exponent in the query term of §8. The argument is two lines: if the codeword u near g disagrees with $\text{Fold}(f, \alpha)$ on at least a δ fraction, each uniform r_i^{shift} independently lands on a disagreement with probability $\geq \delta$, so

$$\Pr[\text{all } t \text{ points miss}] \leq (1 - \delta)^t = \text{the leading term of } \epsilon^{\text{shift}}.$$

So $t_i = \lceil \lambda / -\log_2(1 - \delta_i) \rceil$, and rate reduction saves money precisely because it raises the usable δ and so lowers t . §4’s table and this are two sides of one fact.

What it is in this implementation

In the paper	In the script
sampling r_i^{shift}	<code>challenger::sample(slot)</code> , one base element from the rate
its index in $\mathcal{L}^{(2^k)}$	<code>field::low_bits_to_altstack</code> — these bits are the Merkle path
the 2^k values on the fibre	the opened row, bound to its leaf by <code>merkle::hash_row</code>
$y_i = \text{Fold}(\cdot)(r_i^{\text{shift}})$	<code>multilinear::eval_multilinear(k)</code> folds the row
batching into σ'	<code>constraint::combine_answers(t)</code>
the accumulated pairs	<code>constraint::closing_check</code>

The second row is worth pausing on. **The index is the path** — the sampled bits are directly the direction bits of the Merkle walk, so the spender picks neither which leaf is opened nor where it sits; the binding is structural rather than an added check. It also explains where the depth factor in the cost law comes from: $\text{depth} = \log_2 |\mathcal{L}^{(2^k)}|$, so one shift query is depth Poseidon2 permutations, and that is the origin of the 1300 in §9.

8 Soundness, and what it rests on

Compilation through BCS requires *round-by-round* soundness. WHIR’s Theorem 5.2 gives:

$$\begin{aligned} \varepsilon_{i,s}^{\text{fold}} &\leq \frac{d \cdot \ell_{i,s-1}}{|\mathbb{F}|} + \text{err}^*(\mathcal{C}, 2, \delta_i), & \varepsilon_i^{\text{out}} &\leq \frac{2^{m_i} \ell_{i,0}^2}{2|\mathbb{F}|}, \\ \varepsilon_i^{\text{shift}} &\leq (1 - \delta_{i-1})^{t_{i-1}} + \frac{\ell_{i,0}(t_{i-1} + 1)}{|\mathbb{F}|}, & \varepsilon^{\text{fin}} &\leq (1 - \delta_{M-1})^{t_{M-1}}. \end{aligned}$$

Only $\varepsilon^{\text{shift}}$ and ε^{fin} carry the $(1 - \delta)^t$ term; everything else is field-size noise.

Because WHIR discards quotienting, distance preservation under folding is not enough: the analysis needs *list* preservation, which is the paper’s new tool.

Definition 4.9 (mutual correlated agreement, informal). Gen has mutual correlated agreement if, with high probability over α : for *every* set S on which $\sum_j r_j f_j$ agrees with $\mathcal{C}$, *every* f_i agrees with $\mathcal{C}$ on S .

Standard correlated agreement promises only *some* set T ; equating S and T upgrades “folding preserves distance” to “every codeword close to $\text{Fold}(f, \alpha)$ is the folding of a codeword close to f ” (Theorem 4.20). It is proved in the unique-decoding regime and *conjectured* beyond it (Conjecture 4.12).

Regime	Proximity δ	Status	Verifier hashes	Argument
WHIR-UD	$(1 - \rho)/2$	verified	10 000	621 KiB
WHIR-JB	$1 - \sqrt{\rho} - \eta$	Conj. 4.12(1)	5 100	299 KiB
WHIR-CB	$1 - \rho - \eta$	Conj. 4.12(2)	2 700	156 KiB

WHIR Table 5, $m = 24$, $\rho = 1/2$, $\lambda = 128$, Goldilocks cubic extension.

Part II Implementation review and repair

9 What the crate is, and its cost law

Two crates: `poseidon2` (the permutation, the quartic extension, Merkle verification over KoalaBear) and `whir` (sponge, challenger, sumcheck, multilinear evaluation, constraint, composed verifier). The design premise is sound and well argued — an algebraic hash makes a digest eight field elements, so compressing two children is sixteen stack items handed to an arithmetic routine, and nothing needs concatenating. That removes the `OP_CAT` soft-fork dependency entirely.

Everything else follows from one measured constant: a Poseidon2 permutation is **572 228 bytes** of script, and nothing else is within 1% of it. So

$$C_{\text{script}} \approx N_{\text{perm}} \cdot c_{\text{perm}}, \quad N_{\text{perm}} = \underbrace{\sum_i t_i \cdot \text{depth}_i}_{\text{Merkle}} + N_{\text{transcript}}.$$

For the crate’s example configuration the braced term is $20 \cdot (21+17+13+9) + 20 \cdot 5 = \mathbf{1300}$ permutations. **Every optimisation that matters is an optimisation of that term.**

10 Verified, versus asserted

A test comparing a script against a Rust transcription establishes that the two *agree*; only a test driven by a real proof, with a negative case, establishes that either is *right*.

Component	Evidence	Negative case	Before	Now
Poseidon2, Merkle compression	vs. reference	—	verified	verified
Duplex sponge, padding	vs. reference	short absorb padded $\neq$	verified	verified
Challenger sampling	vs. reference	coefficient order	verified	verified
Merkle opening of a real proof	real commitment root	tampered leaf rejected	verified	verified
Challenge derivation	squeezed in script	perturbed evaluation rejected	D1	fixed
Closing identity	EF identity	each coefficient rejected	D2	fixed
Composed <code>verify()</code>	executed vs. reference	transcript moves the claim	D3	fixed
Row bound to leaf	<code>hash_row</code> vs. Plonky3	every substitution rejected	gap	fixed
Answers batched into target	vs. reference	every answer moves σ'	gap	fixed
Accumulated pairs threaded	—	—	gap	open

11 The three defects

D1 Fiat–Shamir challenges were never derived from the transcript

The challenges fed to the sumcheck chain and the point at which the final polynomial was evaluated were deterministic constants, invented by the test rather than derived by Plonky3.

Consider one round, with $h(1)$ recovered from $h(0) + h(1) = c$:

$$h(r) = (1 - r) c_0 + r (c - c_0) + r(r - 1) c_\infty.$$

Fix any target c' and any $r \notin \{0, 1\}$. Since $r(r - 1) \neq 0$ is invertible,

$$c_\infty := (c' - (1 - r)c_0 - r(c - c_0)) \cdot (r(r - 1))^{-1}$$

reaches c' for *any* c_0 . Composing, a prover supplying the challenges controls the closing claim exactly. **The soundness error of the chain was 1.**

Fix. `sumcheck_round_fs` absorbs the round's (c_0, c_∞) and squeezes r from the resulting rate, so r is a function of the proof and is fixed only after both evaluations are committed. The invariant is `claim(4) || state(16)` with the sponge on top, already the layout `absorb` wants, so rounds compose.

D2 The closing identity was a tautology, in the base field

The test pushed a , b and ab , rotated twice, and `final_check` recomputed ab to compare against ab — true by construction for any proof. It also compared one coefficient, leaving a base-field equality worth $\leq \log_2 P \approx 31$ bits where the quartic extension exists to provide ≈ 124 .

Fix. `final_check` multiplies in EF and compares all four coefficients; the end-to-end weight is copied off the stack rather than pushed, tying the sumcheck chain to the final polynomial.

D3 `verify()` priced a schedule it could not run

`sample_ef` was never called in `src/`; `absorb` leaves a sixteen-element state where `sumcheck_round` expects `claim c0 c_inf r`; and the test only took `.len()`. Two bugs went with it: `open_query` hardcoded rate slot 0, so every query in a round derived the *same* index and opened the *same* leaf; and a round absorbed two elements where it sends two *extension* elements, then paid a second permutation for a squeeze that absorbing had already performed.

Fix. Wired, executed against a reference, with the slot parameterised and the accounting corrected.

12 The repair's stack discipline

```
// in : claim(4) || state(16)      hints pushed above: c0(4) || c_inf(4)
// out: claim'(4) || state'(16)
{ ext4::copy(0) } { ext4::to_altstack() } // park c_inf -- absorb consumes it
{ ext4::copy(1) } { ext4::to_altstack() } // park c0
{ sponge::absorb(8) } // ONE permutation; absorb IS the duplexing
{ challenger::sample_ef(0) } // r := rate[0..4], bound to the transcript
for _ in 0..4 { { 23 } OP_ROLL } // lift claim over state' (2*D + WIDTH - 1)
```



```

{ ext4::from_altstack() } // c0
{ ext4::from_altstack() } // c_inf
for _ in 0..4 { { 15 } OP_ROLL } // lift r to the top (4*D - 1)
{ sumcheck::sumcheck_round() } // consumes claim c0 c_inf r
for _ in 0..16 { { 19 } OP_ROLL } // restore the invariant (D + WIDTH - 1)

```

Each OP_ROLL group lifts the deepest slot of its element first, preserving coefficient order.

These depths are the failure mode of the whole approach. CLAIM_UNDER_STATE_AND_R is $2D + W - 1 = 23$ and not $4D + W - 1 = 31$; the difference between those two is one misread of what sits above the claim, and it produces `InvalidStackOperation` rather than a wrong answer. That is a class of error which appears only when the script is *executed*, which is a footnote to D3: a builder that is measured and never run cannot be wrong in this way, because it is never right in it either.

13 What the repair cost

Rolls and picks are one to two bytes against 572 228 for a permutation, so the wiring is free at this resolution, and the round drops from two permutations to one.

Quantity	Before	Predicted	Measured	Why
Transcript permutations	67	≈ 46	44	absorb is the duplexing
Merkle paths	1300	1300	1300	the term that matters
Transcript share	5.0%	$\approx 3.5\%$	3.3%	—
Tests	—	—	67	all passing

The honest verifier is smaller than the priced skeleton. **There is no efficiency argument for leaving Fiat–Shamir out.** The composed end-to-end script measures **8 967 342 bytes**: a real Plonky3 WHIR proof accepted by one Bitcoin Script execution, with every perturbation of the prover’s evaluations rejected.

14 Constraining the query

Three changes, in the order they had to happen.

The row is bound to its leaf

A query opening has two halves and only one used to be checked. Walking the path proves that *some* committed leaf sits at the sampled index; with the leaf arriving as a hint alongside the row, nothing stopped a spender authenticating the real leaf and folding a different row. `merkle::hash_row` reproduces Plonky3’s `PaddingFreeSponge`: overwrite the first RATE state elements with each chunk and permute, a partial final chunk leaving the rest alone. No padding, no length tag — deliberately *not* the duplex, where zeroing the rest of the rate and binding the count is what stops a short absorb colliding with a padded longer one. Cost: $\lceil \text{row_len}/8 \rceil$ permutations per query.

A round’s answers are batched

`constraint::combine_answers` computes $\sigma' = \hat{h}_k(\alpha_k) + \sum_i \gamma^{i+1} y_i$ by Horner from the top — $t + 1$ multiplications rather than the $2t$ of forming the powers, the outer γ being what shifts every exponent up by one. No permutation at all.

The closing identity, and two readings of it

Each accumulated term collapses, because summing an evaluation against eq *is* evaluation:

$$\sum_b \hat{f}_M(\mathbf{b}) \gamma^e \text{eq}(\mathbf{z}, \mathbf{b}) = \gamma^e \hat{f}_M(\mathbf{z}).$$

So $\hat{w}'$ is exactly the list of $(c_j, \mathbf{z}_j)$ pairs of §2, and checking it means evaluating the final polynomial at each point and taking the weighted sum. That is self-consistent, it is the paper’s reading, and `closing_check` implements it.

It is not the identity Plonky3 checks. There the two polynomials are kept *apart*, each evaluated at its own point:

$$\underbrace{\text{claimed_eval}}_{\text{the chained claim}} = \underbrace{w(\mathbf{R})}_{\text{eval_constraints_poly}} \cdot \underbrace{\hat{f}_M(\mathbf{r}_{\text{fin}})}_{\text{final_evaluations.eval_ext}},$$

and `eval_constraints_poly` evaluates every accumulated $\text{eq}(\mathbf{z}_j, \cdot)$ explicitly. Since this implementation is checked against Plonky3, that is the shape the script must produce — so an eq primitive is needed after all.

The two readings are easy to mistake for one, and the mistake runs in a particular direction: the collapse above is a true statement about the paper, and reading it as a statement about the verifier being targeted removes a primitive that verifier requires. The gap appears only when the two formulations are set side by side.

Measured, since this is where `final_poly_vars` bites:

n_{vars}	bytes per point
2	95,392
3	190,922
4	381,988
5	764,270

Doubling per variable. For the example configuration the accumulated pairs number roughly $\sum_i (t_i + 2) \approx 110$, so at `final_poly_vars` = 4 the closing check is on the order of 42 MB against 743 MB of Merkle work — about 5%. It is also a concrete argument for pushing folding work into earlier rounds: the cost is $n_{\text{points}} \cdot 2^{n_{\text{vars}}}$.

Threading, and why it costs almost nothing

The list of pairs has to reach the closing check having been *derived*, not supplied. The natural guess is that this is expensive: a pair created early should have to be rewritten every round, its coefficient absorbing a factor as the sumcheck binds variables. It does not, and the reason is one line — eq factorises across a split,

$$\text{eq}((\mathbf{z}_{\text{pre}}, \mathbf{z}_{\text{suf}}), (\boldsymbol{\alpha}, \mathbf{y})) = \text{eq}(\mathbf{z}_{\text{pre}}, \boldsymbol{\alpha}) \cdot \text{eq}(\mathbf{z}_{\text{suf}}, \mathbf{y}),$$

so binding a prefix never mixes coordinates. Constraints can simply be *accumulated*, and at the end each is evaluated against the concatenation of every round’s folding randomness,

$$\mathbf{R} = \boldsymbol{\alpha}^{(0)} \parallel \boldsymbol{\alpha}^{(1)} \parallel \dots \parallel \boldsymbol{\alpha}^{(\text{fin})},$$

a constraint of arity n_c reading the **last** n_c **coordinates** of $\mathbf{R}$ — which is what it means for the randomness from round i onwards to be what binds a constraint born at round i .

On a stack that rule is free. Push $\mathbf{R}$ with R_0 deepest and the last n_c coordinates are the *topmost* n_c : every constraint reads from the top, and a longer one reaches further down. No

slicing, no offset arithmetic, no per-round rewriting. Keeping each round’s challenge rather than discarding it costs **76 bytes**, 0.011% of a round.

A second convenience sits in the point expansion. An out-of-domain sample is a scalar u whose constraint point is $(u^{2^{m-1}}, \dots, u^2, u)$; repeated squaring produces u first, so moving the block to the altstack top-first lands the coordinates in exactly the order the eq evaluation consumes them. The two conventions meet without a roll.

Primitive	What it is	Cost
eq_eval	$\prod_i (1 + 2z_i r_i - z_i - r_i)$	$2n$ ext. mult.
expand_univariate	the square-power point	$n - 1$ ext. mult.
constraint_eval	$w(\mathbf{R})$	per constraint, above
sumcheck_rounds_fs_keep	buries each round’s α	76 B per round
constraint_eval_batched	the same, from scalars and χ	+32–45%

None costs a permutation. eq_eval and expand_univariate are checked against Plonky3’s own `Point::eval_eq` and `Point::expand_from_univariate`, not only against a local reference — either could otherwise have been a consistent misreading.

Deriving the constraints, and a convention worth pinning

The list can be *derived* rather than supplied, and far more cheaply than carrying it whole: a round’s constraints are all weighted by powers of one batching challenge, and each of their points is the square-power expansion of a single scalar, so a round is $1 + n$ extension elements rather than $n(1 + \text{arity})$.

The weights needed pinning down. Plonky3’s `Constraint::challenge_powers(shift)` is `shifted_powers( $\chi^{\text{shift}}$ )`, so within a constraint the weights run $\chi^0, \chi^1, \chi^2, \dots$ — starting at **one**. The σ' update in the same protocol uses γ^{i+1} , starting at the challenge itself. **Two conventions inside one protocol** is exactly the sort of thing a script and a reference written by the same hand get wrong together, so the two-constraint case is pinned against the expression written out by hand rather than against either implementation. Starting at one also makes a group a plain polynomial in χ , so Horner evaluates it in $n - 1$ multiplications rather than the $2n$ a power ladder costs.

An absorb four times too narrow

Wiring the threading in exposed a defect the earlier accounting had hidden. Out-of-domain answers and the final polynomial are *extension* elements, and Plonky3 observes them with `observe_algebra_slice` — four base elements each, not one. The out-of-domain points are also sampled one at a time *between* the answers rather than once after them, so a round costs one duplexing per sample and not one per rate block. Both are the kind of error that an accounting cannot catch and an execution can: nothing about the schedule looks wrong until something absorbs a value and the challenge that follows it comes out different.

15 Composing the query openings

The queries were priced and never emitted, and emitting them found two defects and one omission.

D4 open_query could not have run

It took the opening from the main stack and sampled the index with a pick that reaches into the top sixteen slots — so with an opening above the sponge, those picks landed inside the row. The contract could not hold, and nothing caught it because the builder **had no test at all**. It is the same shape as D3: a script that is measured and never executed.

Fix. The opening comes off the altstack and the index is sampled with a pick that reaches past it. The order is forced rather than chosen: the direction bits go on the altstack, so the opening has to come off first or the bits would be popped in its place.

D5 The opening’s root came from the witness

A spender could commit to one tree and open a path in another. Any root with a consistent path passed, and the round’s *absorbed* commitment — the one the transcript is bound to — was never compared against.

Fix. The root is copied from that commitment, which the absorb now keeps. This is the third value in the crate found to be absorbed and then not retained, after the final polynomial and the sumcheck randomness; the pattern is worth naming, because in each case the transcript proved the prover had *said* something and nothing tied what was said to what was later used.

Keeping it needed care about *where*. A sumcheck round buries its challenge directly under the claim, so a commitment parked there would land inside **R** and stop the suffix rule of §14 working. Commitments go below **R**, where they survive the round’s sumcheck rounds — which they must, because Plonky3 opens round i ’s queries against round $i - 1$ ’s commitment, not the one round i just absorbed. Two are alive during a round, the older deeper by exactly one digest.

The omission: the leaf hash was never priced

The row-to-leaf binding of §14 closed a real gap, but the accounting only ever counted the path. At folding factor 4 a row is sixteen extension elements, so sixty-four base elements, so **eight permutations per query** on top of the twenty-one the path costs.

Example configuration	before	now
Merkle paths	1300	1300
leaf hashing	<i>unpriced</i>	800
transcript	54	62
total	1354	2162
query share	—	97.1%

Deriving the constraints, and what it costs

The closing check was sound for the statement and merely *described* everything else: its constraint list came from the witness, so the out-of-domain and shift answers were authenticated and then ignored. They are now derived, and what makes that affordable is that a constraint is far smaller than it looks. A round’s constraints share one batching challenge and each of their points is the square-power expansion of a single scalar, so a round carries $1 + n$ extension elements rather than $n(1 + \text{arity})$. The rounds bury those scalars below the randomness as they sample them and the closing check lifts them back out.

Three things had to line up. **The order:** Horner wants a group’s last term first, and a round produces its scalars in order and its challenge afterwards — so the eq values are built

on the stack as the scalars arrive and Horner runs once the challenge appears. **The shift scalars:** a shift query’s point is g^{index} , and the index is re-sampled rather than kept, because sampling is a pick and the sponge has not moved, while keeping it would mean carrying it past a Merkle walk that uses the altstack. **The arities:** a constraint born at round i is bound by the randomness from round i onwards, so it reads the last $m - r_{\text{len}}$ coordinates — §14’s suffix rule, now with the arities computed rather than passed in.

Closing check	supplied	derived
bytes	1,550,899	92,801,911
share of the verifier	0.126%	7.5%
constraints evaluated	1	110
permutations	0	0

Sixty times larger, and still not a permutation. Everything this report has said about the algebra being cheap survives the composition; what grew, every time, is the hashing.

What is still supplied, and why it always will be. The statement’s own constraint. Its point is public because it *is* the claim being proved, and a different point is a different statement rather than a cheaper proof of the same one. Two fidelity items remain against Plonky3: each group ends with a fresh squeeze for its batching challenge rather than reading whichever rate slot the query indices happened to leave, and Plonky3 folds the statement and the commit phase’s out-of-domain samples into one `initial_constraint` where this treats them as two groups.

16 The stack the verifier keeps

Everything above is about what the verifier computes. This is about where it puts things, which on a stack machine is most of the work and all of the risk: every defect in §15 was a value in the wrong place rather than an arithmetic mistake.

The invariant

Between steps the main stack reads, bottom to top:

$$\underbrace{\text{carried scalars}}_{\text{constraints}} \parallel \underbrace{\hat{f}_M}_{2^v \text{ elements}} \parallel \underbrace{\mathbf{R}}_{m \text{ elements}} \parallel \underbrace{\text{claim}}_1 \parallel \underbrace{\text{sponge}}_{16 \text{ slots}}.$$

Three constraints fix that order, and between them there is no freedom left.

The sponge is on top because absorbing wants its inputs directly above the state and sampling is a pick into the top sixteen slots. Everything else has to be underneath, which is why a query opening — the one thing that must sit above the state while it is checked — needed a sampling primitive that reaches past it.

$\mathbf{R}$ must be contiguous, because §14’s rule reads a constraint’s local point as a *suffix* of it. A sumcheck round appends its challenge directly under the claim, so anything else parked there would land in the middle of the randomness and the suffix rule would silently read the wrong coordinates. That single requirement is what forces commitments and the final polynomial below $\mathbf{R}$ rather than under the claim, and it is not a preference: nothing would fail loudly if it were violated.

The altstack is the transcript. Everything the spender supplies is on it in consumption order, so nothing may be parked there across a step. A value needing to survive one has to be buried on the main stack instead — which is the whole reason the layout above exists.

Why every depth is a constant

Burying a value under a region, and copying one back out from under it, are both loops at a *fixed* depth. The argument is the same in both directions and worth stating once, because it is used a dozen times and looks like an off-by-one every time:

Rolling the deepest slot of a region to the top removes one item from below the cursor and adds one above it. The two cancel, so the next-deepest slot is at the same depth as the last one was.

So “lift a region of k slots over a block of b ” is k rolls at depth $b + k - 1$, and it preserves the region’s internal order. The same cancellation is why `ext4::copy` is four picks at one depth, and why copying a block back out is b picks at the depth the region left it.

The one place this does *not* hold is dropping from the bottom: each slot removed shrinks the stack, so the depth decreases by one per step. Both forms appear, and confusing them is exactly the class of error that only shows up when the script is executed.

What a round does to the layout

Step	Effect on the layout	Depth
absorb the commitment	one segment below $\mathbf{R}$	$4r + 20 + 8 - 1$
out-of-domain sample	one element below $\mathbf{R}$	$4r + 20 + 4 - 1$
query opening	nothing; balanced	reaches the commitment
shift scalar	one element below $\mathbf{R}$	$4r + 20 + 4 - 1$
batching challenge	one element below $\mathbf{R}$	$4r + 20 + 4 - 1$
drop the opened commitment	a segment leaves the middle	recomputed
sumcheck rounds	$\mathbf{R}$ grows under the claim	$2 \cdot 4 + 16 - 1$

Only the second-to-last row is awkward. Dropping a commitment removes a segment from the *middle* of the carried region, so every depth above it shifts — which is why the generator keeps a model of the region rather than hard-coding the numbers. Two commitments are live at once, because a round’s queries open the *previous* round’s codeword while the current one has already been absorbed.

And what the closing check does with it

In order: drop the sponge; lift $\hat{f}_M$ over $\mathbf{R}$ and the claim and collapse it to one value; evaluate the statement’s constraint, which *reads* $\mathbf{R}$ without consuming it; lift the carried scalars out and hand them to the altstack; evaluate the accumulated constraints, which consumes $\mathbf{R}$; and compare.

The order is not free. The statement has to be evaluated before the scalars reach the altstack, because its pairs are down there underneath them; and it has to be evaluated before the accumulated constraints, because those consume the randomness it needs. Moving the scalars reverses their order twice — once rolling them up, once moving them across — which is what lets a round bury them in the order it samples them and still have Horner read them in the order it wants.

17 Two unused levers

Proof of work

`pow_bits`: 0 is why the security level tops out at 100. Both reference implementations grind: STIR’s benchmarks take 22 bits, reducing the soundness required from the IOP from 2^{-128} to

2^{-106} ; WHIR uses $m - \log \rho - 3$, also 22 at $m = 24$, $\rho = 1/2$. The trade is unusually favourable here — a query costs $\text{depth} \times 572 \text{ KB}$, a proof-of-work check costs one permutation regardless of depth:

$$\Delta N_{\text{perm}} = -\lceil 20/4 \rceil \cdot 9 + 1 = -44 \text{ of } 234 \approx \mathbf{19\% \text{ of the dominant term.}}$$

Soundness regime

`SecurityAssumption::CapacityBound` is the cheapest of the three and the only one resting on the unproven half of Conjecture 4.12. Nothing in the implementation mentions a soundness assumption: the script executes a schedule, and the regime reaches it only as a query count. So the choice is a configuration change, and `whir/tests/budget.rs` prices it — each regime given the grinding it needs, since holding `pow_bits` fixed reports *not derivable* for the weaker assumptions rather than a price.

Regime	m	pow	queries	permutations	vs CB
UD, proved	16	0	110	7 376	7.18 ×
JB, Conj. 4.12(1)	16	26	39	1 853	1.80×
CB, Conj. 4.12(2)	16	19	21	1 027	1.00×
UD, proved	20	0	110	11 171	9.01 ×
JB, Conj. 4.12(1)	20	33	35	2 222	1.79×
CB, Conj. 4.12(2)	20	28	19	1 240	1.00×

Two things the paper’s own comparison does not show at this instance. **UD wants no grinding at all**, paying entirely in queries, and its count is capped at 110 regardless of size — so the gap widens with m rather than staying fixed. And **JB wants more grinding than CB**, 26 bits against 19, which is the direction a weaker assumption should push: less can be assumed, so more has to be bought.

A defensible choice, and the papers’ own benchmarking default. But the proved alternative costs 7 to 9 times here, not the 3.7 of WHIR’s Table 5 — that table is Goldilocks with a cubic extension at $m = 24$ and rate $1/2$, and a ratio does not travel between instances. A verifier that settles bitcoin should say out loud which assumption it stands on, and what the proved one would cost *here*.

Part III Completeness and soundness

This part separates two things: the completeness and soundness of the **protocol** (the papers' results, stated here and explained in shape), and of **this script** (this repository's responsibility, covering only what it actually checks). Conflating them is the error this report has repeatedly pointed at.

18 Completeness

Proposition (protocol). If $f \in \text{CRS}[\mathbb{F}, \mathcal{L}, m, \hat{w}, \sigma]$ and the prover is honest, the verifier accepts with probability 1.

Proof sketch. Each equality is an identity in the honest case, not a probabilistic event.

1. **Sumcheck.** The honest $\hat{h}_\ell(X) = \sum_{\mathbf{b}} \hat{w}(\hat{f}(\boldsymbol{\alpha}, X, \mathbf{b}), \dots)$ makes $\hat{h}_1(0) + \hat{h}_1(1) = \sigma$ and $\hat{h}_\ell(0) + \hat{h}_\ell(1) = \hat{h}_{\ell-1}(\alpha_{\ell-1})$ hold by the definition of the sum.
2. **Folding preserves codewords.** By Claim 4.15, $f \in \text{RS}$ implies $\text{Fold}(f, \boldsymbol{\alpha}) \in \text{RS}$ with multilinear extension $\hat{f}(\boldsymbol{\alpha}, \cdot)$, so the honest g is a codeword of the right degree on the right domain.
3. **Answers.** The honest prover sets $y_0 = \hat{g}(\mathbf{z}_0)$, and y_i is the Fold value the verifier computes from f itself; the two agree by construction.
4. **Recursive claim.** Substituting (2) and (3) into the definitions of $\hat{w}'$ and σ' gives $\sum_{\mathbf{b}} \hat{w}'(\hat{g}(\mathbf{b}), \mathbf{b}) = \sigma'$, so the inductive hypothesis is preserved.
5. **Closing.** $\hat{f}_M$ is sent in the clear and both final checks hold by construction. $\square$

Proposition (script). Given a proof Plonky3 accepts and a witness laid out as documented, the script runs to completion without aborting.

Why this is measured, not argued. `whir/tests/end_to_end.rs` produces a real proof with Plonky3's prover, confirms it with Plonky3's *own* verifier, and only then hands the same object to the script. The order is deliberate: comparing a script against a Rust reference alone would let both pass while misreading a proof in the same way.

19 Soundness

Protocol: round by round

BCS/Fiat-Shamir needs *round-by-round* soundness, not plain soundness: a state function $\text{State}(\text{tr}) \in \{0, 1\}$ that is 1 on the empty transcript iff the statement is true, that a prover cannot flip back once 0, that a verifier's next challenge flips with probability $\leq \varepsilon_i$, and whose being 0 on a full transcript implies rejection. The strengthening is necessary because Fiat-Shamir lets a prover retry challenges, under which plain soundness does not survive.

The four errors of §8 split in two, and the split is useful to an implementer:

Field-size terms ($\varepsilon^{\text{fold}}$, ε^{out} , and the second term of $\varepsilon^{\text{shift}}$): of the form $\text{poly}(2^m, 1/\rho)/|\mathbb{F}|$, shrunk by *extending the field*.
Query terms (the leading $\varepsilon^{\text{shift}}$ and ε^{fin}): $(1 - \delta)^t$, shrunk only by *querying more*, and on Bitcoin each query is depth permutations.

The whole cost structure sits on that line: field size is free (the quartic extension of KoalaBear gives about 124 bits), query count is the only thing that is paid for, and rate reduction exists to lower it.

Script: what it actually guarantees

Property checked	Ground	Error
Challenges unchooseable	r squeezed <i>after</i> (c_0, c_∞) absorbed	see next
One sumcheck round	false claim $\Rightarrow h$ not identical; Schwartz–Zippel	$\leq 3/2^{124}$
Opening is committed	path recomputes the root, <code>OP_EQUALVERIFY</code>	collision
Folded row is the committed row	<code>hash_row</code> reproduces <code>PaddingFreeSponge</code>	preimage
Index unchooseable	index bits <i>are</i> the path direction bits	—
Answers cannot cancel	random linear combination in powers of γ	$\leq (t+1)/2^{124}$
Closing identity	all four EF coefficients; weight taken from the stack	$\leq 1/2^{124}$

Not covered, and so not in that table: threading the accumulated (weight, point) pairs between rounds. Both ends exist — σ' is computed and the closing identity is checked — but the pairs are not yet carried from round to round, so what the script guarantees is that each step holds individually, not WHIR’s full round-by-round soundness. The gap between §8’s ε and this table is exactly that.

An assumption that must be named. The query-term errors above are under the capacity bound (`SecurityAssumption::CapacityBound`), which rests on the *unproven* half of Conjecture 4.12. Under the proved unique-decoding setting this implementation costs 7 to 9 times as much — measured at §17, not read off the paper’s table, which is a different instance — and script bytes scale with the query count.

Why D1 was “soundness error = 1”

Worth isolating, because it is the line between the first row of that table holding and not. Before the fix r came from the witness, so the prover could see r before choosing (c_0, c_∞) — or choose all three together. For any target c' and any $r \notin \{0, 1\}$,

$$h(r) = (1 - r)c_0 + r(c - c_0) + r(r - 1)c_\infty$$

is an **affine** map in c_∞ whose coefficient $r(r - 1) \neq 0$ is invertible, hence **surjective** onto EF: pick any c_0 and solve for the unique c_∞ that hits c' . Schwartz–Zippel does not apply — it bounds “fix a polynomial, then sample a random point”, and here the point was not random. Composing across rounds, the closing claim is entirely the prover’s, and the chain’s soundness error is 1 rather than something small. That is why the fix had to bind r to the transcript rather than choose a better constant.

Sources: G. Arnon, A. Chiesa, G. Fenzi, E. Yogev, *STIR: Reed–Solomon Proximity Testing with Fewer Queries*, ePrint 2024/390 (CRYPTO ’24); and *WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification*, ePrint 2024/1586. Implementation figures are from the test suite: `verifier_cost`, `end_to_end`, `verifier`, `constraint` and `budget`. The 2162 permutations, 2.9% transcript share, 8 967 342-byte composed sumcheck chain, per-constraint closing-check sizes and 98 passing tests are all measured on this machine. What a verifier would cost on-chain, and the chunking a disprove needs, are the repository README’s subject rather than this document’s.